

Scraper Module — Nepal Supreme Court Hearing Tracking

daily + weekly cause lists → normalize → store → match → notify → weekly CSV snapshot

com.lawfirm.erp.modules.scraper · Spring Boot 4 · jsoup + java.net.http

1. What the module does

Every weekday the module pulls the **entire cause list** of each court where we have an active client case, stores it, then **matches our registered cases** against it and notifies. Every Monday it also pulls the **weekly (Sun–Fri)** list, and exports the previous completed week to a **CSV file** as an offline snapshot. The DB is the system of record; the CSV is a convenience copy — nothing is ever deleted.

2. The pipeline



How one court is scraped

1. **GET** the endpoint first — the server issues `PHPSESSID`, `court_session` (which binds the User-Agent, so a fixed browser UA is sent) and an F5 `TS...` cookie.
2. **POST** the form: daily = `todays_date` (Arabic BS) + `pesi_date` (Devanagari BS) + `submit`; weekly = `pesi_bar=all` + `submit`.
3. The server answers **302** with a flash message in the cookie; we follow `Location` with a GET to receive the final page.
4. Parse the page with the two dedicated parsers, normalize, and **upsert**.

All numerals on the site are Devanagari (०८१-८४-३८२७) — converted to Arabic on ingest. Dates are Bikram Sambat; both the BS string and the converted AD date are stored (`NepaliDateUtil` , verified: 2083-05-01 = 2026-08-17). Every row carries both case-number forms: the court-visible `caseNoBs` (०८१-८४-३८२७) and the court-scoped `caseNoInternal` (39-०८१-३२०३० — the parenthesized form, which is what matching uses).

Not published? If the site has no list for a date (common early morning), the scrape returns 0 rows, nothing is stored for that date, and an alert is logged. This is expected behaviour, not an error.

3. Storage (PostgreSQL, auto-created)

Table	Contents	Key / notes
<code>scraper_courts</code>	Court registry (39 = Kathmandu, 63 = Gulmi)	<code>courtId</code> PK, matches the site URL segment
<code>scraper_client_cases</code>	Your cases: <code>courtId</code> + <code>caseNoInternal</code> + status	ACTIVE rows drive which courts are scraped
<code>scraper_daily_hearings</code>	Normalized daily rows (has outcome column)	unique (<code>courtId</code> , <code>caseNoInternal</code> , <code>hearingDateBs</code>) → idempotent upsert
<code>scraper_weekly_hearings</code>	Normalized weekly rows (no outcome column)	same unique key
<code>scraper_hearing_matches</code>	The app's working set: your case × hearing, with full detail	unique per (<code>clientCaseId</code> , <code>courtId</code> , <code>hearingDateBs</code>); notified flag

Matching

After every ingest, `HearingMatchingService` joins `client_cases` × `hearings` on (`courtId`, `caseNoInternal`) for today/upcoming hearings. New matches are inserted into `hearing_matches` (deduped — re-runs skip existing), copying the **full row** (subject, plaintiff, defendant, `caseNoBs`, judge, order type) so matches are self-contained. Legacy matches created before the detail columns existed are backfilled automatically on startup.

Notification

New matches go through the `NotificationDispatcher` interface — currently the log stub (`[scraper-notify:log]` ...). Delivery is at-least-once: a match is marked `notified` only after a successful dispatch, so a broken channel never loses a match and never blocks matching.

4. Weekly CSV export (new)

Every Monday **10:30** (`scraper.export-cron`), `HearingExportService` writes the previous **Monday–Sunday** window (daily + weekly rows merged, same normalized schema) to:

```
export-dir/week-<monday>_<sunday>.csv      e.g. ./exports/week-2026-08-10_2026-08-16.csv
```

- **UTF-8 with BOM** — Excel renders the Devanagari text correctly (a CSV without it opens as mojibake).
- **All fields quoted** — party names and subjects contain commas, so quoting is mandatory.
- **matched column** — `Y` / `N` flag marking rows that appear in `hearing_matches` (i.e. one of our cases).
- **Columns**: `courtId`, `hearingDateBs`, `hearingDateAd`, `caseNoBs`, `caseNoInternal`, `judgeName`, `subject`, `plaintiff`, `defendant`, `orderType`, `source`, `matched`, sorted by court then date.
- **Idempotent** — same file name per week; writes to a `.tmp` then atomically renames (never a half-written file). Re-running overwrites.
- The export is **independent of the scrape** — it only reads the DB, so a failed scrape still exports whatever exists.

Configurable: `scraper.export-dir` (default `./exports`). Directory is created if missing.

5. Endpoints (frontend-facing)

GET

`/api/v1/cases/{caseNo}/hearing-status`

DB only · no live scrape

Upcoming + past hearings for a court-scoped internal case number (e.g. `39-081-32030`). Roles: `FIRM_ADMIN` / `ADVOCATE` / `PARALEGAL`.

```
GET /api/v1/cases/39-081-32030/hearing-status

{
  "success": true, "responseCode": 200,
  "data": {
    "caseNoInternal": "39-081-32030",
    "caseNoBs": "081-C4-3827",
    "courtId": 39,
    "courtName": "काठमाडौं जिल्ला अदालत",
    "upcoming": [
      { "hearingDateAd": "2026-08-18", "hearingDateBs": "2083-05-02",
        "judgeName": "माननीय जिल्ला न्यायाधीश श्री अशोककुमार बस्नेत",
        "subject": "लेनदेन", "orderType": "स्थगित", "source": "DAILY" }
    ],
    "history": [ ]
  }
}
```

```
}  
}
```

POST`/api/v1/scrapper/admin/scrape?courtId=&date=`**FIRM_ADMIN only**

Manual single-court daily scrape for backfill/testing. `date` is BS yyyy-mm-dd (defaults to today).

```
POST /api/v1/scrapper/admin/scrape?courtId=39&date=2083-05-01
```

```
{ "success": true, "data": { "courtId": 39, "success": true, "rows": 672, "error": null } }
```

POST`/api/v1/scrapper/admin/export`**FIRM_ADMIN only · new**

Regenerate the weekly CSV export for the previous completed Monday–Sunday window (overwrites the file). Returns the absolute file path.

```
POST /api/v1/scrapper/admin/export
```

```
{ "success": true, "data": "G:/.../exports/week-2026-08-10_2026-08-16.csv" }
```

6. Scheduled jobs

Job	Cron (overridable)	What it does
Daily scrape	<code>scraper.daily-cron</code> — 10:05 Mon–Fri	Every active court → today's daily list → upsert → match + notify
Weekly scrape	<code>scraper.weekly-cron</code> — 10:15 Mon	Every active court → Sun–Fri weekly list → upsert → match + notify
CSV export	<code>scraper.export-cron</code> — 10:30 Mon	Previous Mon–Sun window → <code>week-....csv</code>

All jobs no-op when `scraper.enabled=false`. Also note: adding `@EnableScheduling` activated the case-management module's lapsed-appeal watcher, which was previously dormant.

7. Frontend consumption guide

1. **Register a case** — add a row to `scraper_client_cases` (courtId + caseNoInternal + ACTIVE). No CRUD endpoint exists yet; until one is added, register via DB/seed. Once registered, the court is scraped automatically.
2. **Show a case's hearing status** — call `GET /api/v1/cases/{caseNoInternal}/hearing-status` whenever a case screen opens. Fast, DB-only, shows upcoming + history with judge, subject, outcome and source.
3. **Notifications** — new matches are logged (`[scraper-notify:log]`) and stored in `hearing_matches` (field `notified`). A future real channel (email/SMS/webhook) plugs into the `NotificationDispatcher` interface; until then, poll the hearing-status endpoint or read matches.
4. **Weekly snapshot** — the CSV file is a human-facing artifact (open in Excel; Nepali text renders because of the BOM). Not needed by the frontend unless you add a download link to `POST /api/v1/scraper/admin/export` .

8. Configuration

```
scraper:
  enabled: true           # master switch
  base-url: https://supremecourt.gov.np
  request-delay-ms: 1500  # politeness delay between court-site requests
  connect-timeout-ms: 15000
  read-timeout-ms: 60000
  max-concurrent-courts: 5 # fan-out width
  export-dir: ./exports    # weekly CSV location
  daily-cron: "0 5 10 * * MON-FRI"
  weekly-cron: "0 15 10 * * MON"
  export-cron: "0 30 10 * * MON"
```